

Practical - 4

WEB DEVELOPMENT FRAMEWORKS (ITUE203)

SEMESTER: 3RD, 2026-27

ID: 25DCE110

Practical Title: JavaScript DOM Manipulation, Event Handling, and UI Interactivity

Problem Definition: Add dynamic UI components such as collapsible FAQ, modal popup, image/content slider, notification banner, hamburger menu, and light/dark theme switcher using JavaScript

Theory

1. How are DOM elements selected and modified?

DOM elements are selected using methods such as `getElementById()` and `querySelectorAll()`. New elements are created using `createElement()`, while their content is modified using `innerText` and `innerHTML`. CSS styles are also modified dynamically using the `style` property.

2. Are event listeners attached correctly?

Yes, event listeners are attached correctly using `addEventListener()`. They are used to handle events such as button clicks and certificate link clicks. Functions such as click events allow the webpage to respond to user interactions.

3. Is localStorage used for remembering theme choice?

Yes, `localStorage` is used to store the selected theme as either "dark" or "light". When the page loads, JavaScript checks the stored value and automatically applies the previously selected theme.

4. Does interactivity improve usability without breaking accessibility?

Yes, the interactive features improve usability by providing navigation menus, notifications, theme switching, and confirmation messages. The controls remain clickable and understandable, and the implementation does not remove the basic functionality of the webpage.

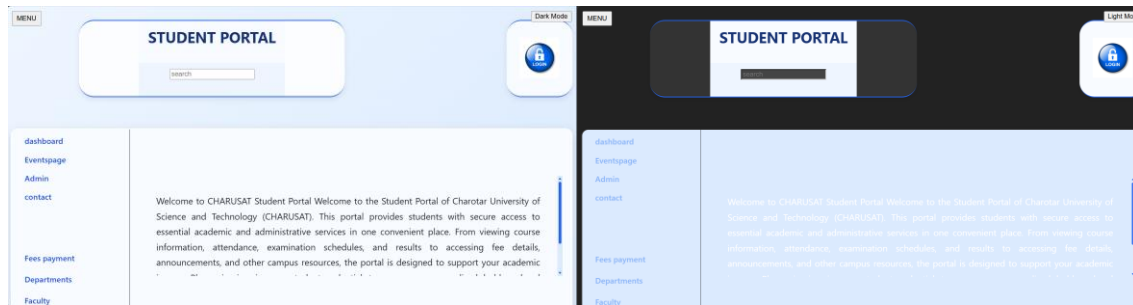
A. Implementation/ Output/Result (Keep only one option which is suitable to practical)

Function / Method	Purpose
<code>document.addEventListener()</code>	Runs code when an event occurs, such as page loading
<code>DOMContentLoaded</code>	Detects when the HTML document has finished loading
<code>document.createElement()</code>	Creates a new HTML element
<code>document.getElementById()</code>	Selects an element using its ID
<code>document.querySelectorAll()</code>	Selects multiple elements
<code>element.innerText</code>	Changes or reads text inside an element
<code>element.innerHTML</code>	Adds HTML content inside an element
<code>element.style</code>	Changes CSS styles using JavaScript
<code>document.body.appendChild()</code>	Adds an element to the page
<code>document.body.insertBefore()</code>	Inserts an element before another element
<code>element.addEventListener()</code>	Attaches an event handler
<code>setInterval()</code>	Repeats code after a fixed time interval
<code>setTimeout()</code>	Executes code after a specified delay
<code>element.remove()</code>	Removes an element from the webpage

Function / Method	Purpose
alert()	Displays an alert message
confirm()	Displays OK/Cancel confirmation dialog
window.open()	Opens a page/file in a new browser tab/window
event.preventDefault()	Prevents the default action of an event
localStorage.getItem()	Retrieves stored data from browser storage
localStorage.setItem()	Stores data in browser storage
Array.length	Gets the number of items in an array
forEach()	Performs an operation for every item in a collection
includes()	Checks whether a string contains specified text

B. Evidence of Work (Screenshots):

Dark/Light Theme Switcher:



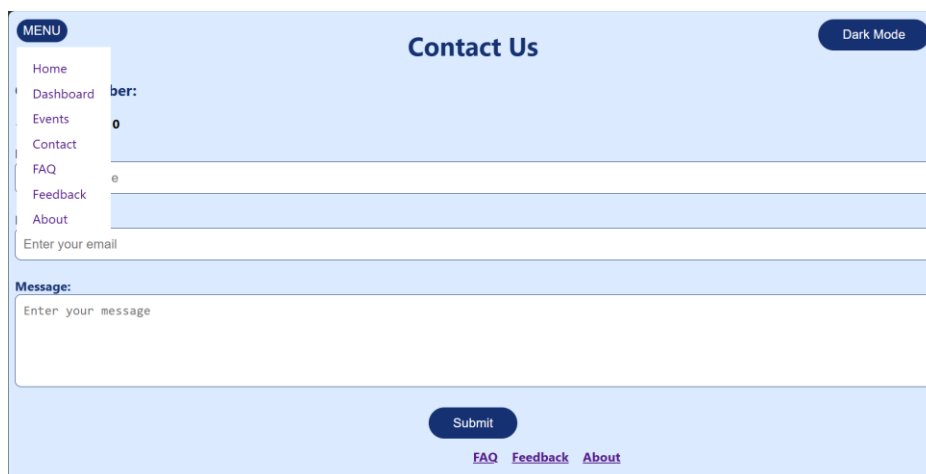
Light theme

Dark theme

Description:

This JavaScript code creates a Dark Mode button that allows the user to switch between dark and light themes. The `makeDark()` function changes the background, text, tables, input fields, and links to dark-theme styles, while the `makeLight()` function restores the original appearance. The selected theme is stored in `localStorage`, so the browser can remember the user's theme preference even after refreshing or reopening the page. This demonstrates DOM manipulation, event handling, and browser storage.

Hamburger Menu:

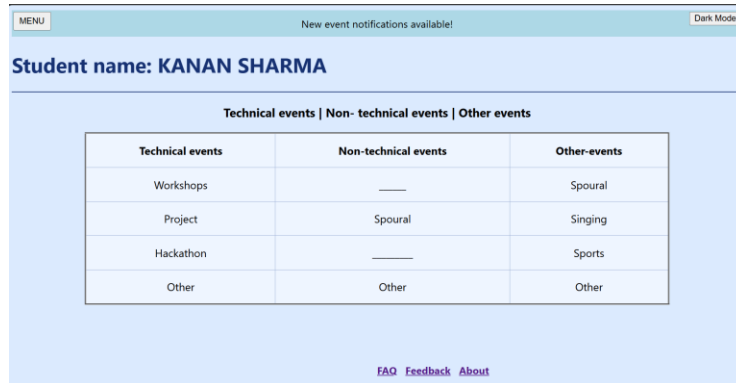


Description:

This JavaScript code creates a hamburger-style navigation menu dynamically. A MENU button is created and positioned at the top-left corner of the webpage. When the user clicks the button, the navigation menu is displayed, and clicking it again hides the menu.

The menu contains links to different pages such as Home, Dashboard, Events, Contact, FAQ, Feedback, and About. This demonstrates DOM element creation and click event handling.

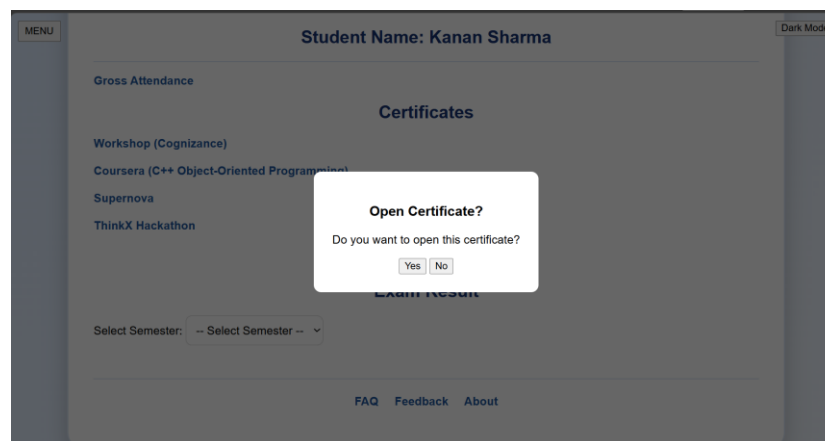
Notification Banner:



Description:

This JavaScript code creates a temporary notification banner that informs the user that new event notifications are available. The banner is dynamically created using `createElement()` and styled using JavaScript. It is inserted at the beginning of the webpage using `insertBefore()`. The `setTimeout()` function automatically removes the notification after 5 seconds. This demonstrates how JavaScript can create and remove DOM elements dynamically.

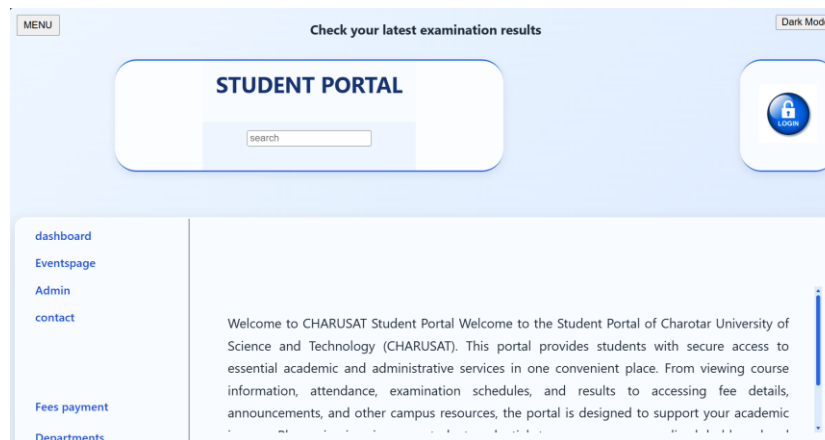
Certificate Confirmation:



Description:

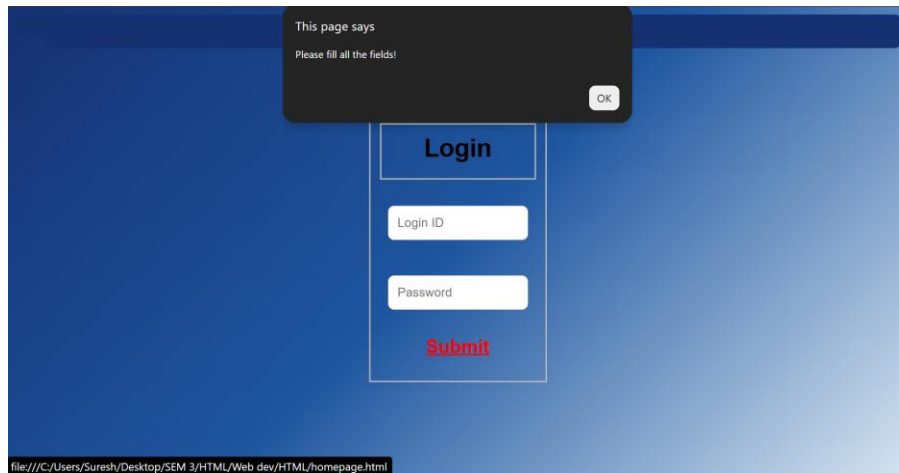
This JavaScript code adds a confirmation step before opening certificate files. It selects all anchor (<a>) elements on the webpage and checks whether their links contain the word "Certificates". For certificate links, a click event is added. The normal link action is first stopped using preventDefault(), and a confirm() dialog asks the user whether they want to open the certificate. If the user selects OK, the certificate is opened in a new browser tab using window.open().

Announcement Banner:



Description:

This JavaScript code creates a dynamic announcement banner at the top of the webpage. It stores multiple announcements in an array and displays them one by one. The setInterval() function changes the announcement every 2 seconds, and when the last announcement is reached, it starts again from the first one. The banner is created dynamically.

Login Validation:**Description:**

This JavaScript code performs basic validation of a login form. The `checkLogin()` function retrieves the Login ID and password entered by the user using `getElementById()`. It checks whether either field is empty. If any field is empty, an alert message is displayed and the form submission is stopped by returning false. If both fields contain values, a login successful message is displayed and the function returns true. This provides basic client-side form validation.

C. GitHub Repository Link:

<https://github.com/kanan2007s-sys/Web-development-/tree/main/JS>

D. Challenges Faced:

I faced many challenges like finding the correct functions. How to make the javascript files work with the HTML and CSS code.

E. Learning Outcomes:

I learned how to use Javascript at beginner level.

Faculty Remarks: